

Claude Code Mastery: Ship Apps in 48 Hours

Product Overview

A 5-module course that takes you from zero to shipping revenue-generating apps using Claude Code. No CS degree. No bootcamp. No \$10K in courses. Just a terminal, Claude, and 48 hours.

Pricing

\$47 (one-time)

Target Audience

- Solo developers who want to ship faster than teams of 5
- Non-technical founders who keep paying \$5K-\$15K for MVPs that take 3 months
- Freelancers billing \$50/hr who want to bill \$200/hr by shipping 4x faster
- Anyone who’s watched tutorials for 6 months and still hasn’t launched anything

What’s Inside

Module	Title	Outcome
1	Setup & CLAUDE.md	Your dev environment configured in 20 minutes, with a CLAUDE.md that makes Claude build YOUR way
2	First App	A working PWA built, tested, and functional in 2 hours
3	Deployment	Your app live on a real domain with SSL, for \$0-\$12/year
4	Monetization	Stripe payments integrated, paywall logic working, first dollar collected
5	Portfolio	A system for shipping 10+ apps, knowing which to kill, which to scale

Guarantee

If you follow Module 2 and don't have a working app in 2 hours, email for a full refund. No questions.

Format

- 5 modules, each 800-1500 words
- Real code examples (copy-paste, not pseudocode)
- Real tool names with real pricing
- Zero filler, zero “mindset” sections, zero fluff

Module 1: Getting Started with Claude Code

What You'll Have After This Module

A fully configured development environment with Claude Code installed, a CLAUDE.md file that acts as your persistent project brain, and hooks that enforce your standards automatically. Total time: 20 minutes.

Step 1: Install Claude Code (3 minutes)

Open your terminal. Run:

```
npm install -g @anthropic-ai/claude-code
```

That's it. No Docker. No virtual environments. No config files to hunt down.

Verify it works:

```
claude --version
```

You need an Anthropic API key. Go to console.anthropic.com, create an account, add \$20 in credits (this will last you weeks of heavy building), and grab your key.

Set it:

```
export ANTHROPIC_API_KEY=sk-ant-your-key-here
```

Add that line to your ~/.zshrc or ~/.bashrc so it persists across terminal sessions.

Step 2: Create Your First Project (2 minutes)

```
mkdir my-first-app && cd my-first-app
claude
```

Claude Code is now running inside your project directory. Everything it creates goes here. Every file it reads comes from here. It's sandboxed to your project — it won't touch your system files.

Type `/init` to generate a starter `CLAUDE.md`. But don't use the default one. Replace it with what's in Step 3.

Step 3: The CLAUDE.md That Actually Works (10 minutes)

Your `CLAUDE.md` is the single most important file in your project. Claude reads it at the start of every session. It's your persistent memory, your coding standards, your architectural decisions — all in one file.

Here's the template that ships products:

```
# Project: [YOUR APP NAME]

## What This App Does
[One sentence. If you can't describe it in one sentence, you don't
  know what you're building.]

## Tech Stack
- Frontend: Vanilla JS + HTML + CSS (no frameworks until you need
  them)
- Backend: None (start static, add Supabase when you need auth/data)
- Hosting: Surge.sh (free, instant deploys)

## Architecture Rules
1. Every feature goes in its own file. No 500-line index.js files.
2. CSS uses BEM naming. No inline styles.
3. All user-facing text lives in a constants.js file (makes i18n easy
  later).
4. Error handling on every fetch call. No silent failures.
5. Mobile-first. Every layout starts at 320px and scales up.

## Current Status
- [x] Project initialized
- [ ] Core feature: [describe it]
- [ ] Stripe integration
- [ ] Deploy to production

## Commands
- `surge ./ my-app-name.surge.sh` - deploy
- `npm serve .` - local dev server
```

Do NOT

- Add TypeScript (overhead isn't worth it for MVPs)
- Add React/Vue/Svelte (vanilla JS ships faster for simple apps)
- Create a database until we have 10+ users
- Write tests until the core loop is validated with real users

Save this as `CLAUDE.md` in your project root. Every time you start a Claude Code session, it reads this file first. Update it as your project evolves.

The key insight most people miss: **CLAUDE.md is not documentation. It's a control mechanism.** The "Do NOT" section is where the magic happens. Without it, Claude will add TypeScript, install 14 npm packages, and architect a microservices backend for your landing page.

Step 4: Set Up Hooks (5 minutes)

Hooks run automatically before or after Claude takes actions. They're your guardrails.

Create the hooks directory:

```
mkdir -p .claude/hooks
```

Create a pre-commit hook that prevents Claude from adding dependencies you didn't approve:

```
cat > .claude/hooks/pre-tool-use.sh << 'EOF'
#!/bin/bash
# Block npm install unless explicitly approved
if echo "$CLAUDE_TOOL_INPUT" | grep -q "npm install"; then
    echo "BLOCKED: npm install detected. Add the package to CLAUDE.md
    approved list first."
    exit 1
fi
EOF
chmod +x .claude/hooks/pre-tool-use.sh
```

Create a post-session hook that auto-updates your CLAUDE.md status:

```
cat > .claude/hooks/post-session.sh << 'EOF'
#!/bin/bash
# Log what changed this session
echo "## Session $(date +%Y-%m-%d_%H:%M)" >> .claude/session_log.md
git diff --stat >> .claude/session_log.md 2>/dev/null
EOF
chmod +x .claude/hooks/post-session.sh
```

Configure hooks in your `.claude/config.json`:

```
{
  "hooks": {
    "pre-tool-use": ".claude/hooks/pre-tool-use.sh",
    "post-session": ".claude/hooks/post-session.sh"
  }
}
```

```
}  
}
```

What You Should Have Now

1. Claude Code installed and authenticated
2. A project directory with a `CLAUDE.md` that controls Claude's behavior
3. Hooks that prevent scope creep and log your progress

If any of these aren't working, don't move to Module 2. Fix them now. Every minute you spend on setup saves 10 minutes of debugging later.

Common Mistakes to Avoid

Mistake 1: Using Claude Code without `CLAUDE.md`. Claude will make reasonable decisions, but "reasonable" means "generic." Your `CLAUDE.md` makes it make YOUR decisions.

Mistake 2: Making `CLAUDE.md` too long. Keep it under 100 lines. Claude reads the whole thing every session. If it's 500 lines, you're wasting tokens and diluting the important instructions.

Mistake 3: Not updating `CLAUDE.md`. After every major decision (chose Supabase over Firebase, decided to add dark mode, pivoted the pricing), update `CLAUDE.md`. It's your project's source of truth.

Mistake 4: Treating Claude Code like ChatGPT. You don't paste code into it and ask for fixes. You tell it what to build, and it creates, edits, and runs files directly on your machine. It's a coworker with terminal access, not a chatbot.

Next module: You'll build a complete PWA in 2 hours using everything you just set up.

Module 2: Build Your First PWA in 2 Hours

What You'll Have After This Module

A fully functional Progressive Web App — installable on phones, works offline, looks professional — built from a single Claude Code session. Not a tutorial project. A real app you could charge money for.

The App: A Habit Tracker That Actually Works

We're building "StreakPad" — a habit tracker that stores data locally, works offline, and installs like a native app. Habit trackers sell. There are 47 of them on the App Store charging \$4.99-\$9.99/month. You're going to build one in 2 hours for \$0.

Why a habit tracker? Three reasons: 1. Simple data model (habits + daily check-ins = two arrays) 2. High perceived value (people pay \$60/year for these) 3. Zero backend needed (localStorage handles everything until you have 1000+ users)

Hour 1: Core Functionality (0:00 - 1:00)

Open your terminal in your project directory and launch Claude Code:

```
cd streakpad && claude
```

Give Claude this prompt. Copy it exactly:

Build a Progressive Web App called StreakPad — a habit tracker.

Requirements:

- Single page app, vanilla JS, no frameworks
- Users can add habits with a name and target frequency (daily, weekly)
- Main screen shows today's habits with tap-to-complete toggles
- Streak counter shows consecutive days completed for each habit
- All data stored in localStorage
- Dark mode by default with a light mode toggle
- Mobile-first, max-width 480px centered on desktop
- PWA manifest and service worker for offline + installable
- Color scheme: #0A0A0A background, #00FF88 accent, #FFFFFF text

File structure:

- index.html (entry point)
- css/style.css
- js/app.js (main logic)
- js/storage.js (localStorage wrapper)
- manifest.json
- sw.js (service worker)
- icons/ (generate simple SVG icons, 192x192 and 512x512)

Start with the data layer (storage.js), then UI, then PWA features.

Claude will create all these files. Watch the output — it's building your app in real-time, file by file. This takes about 10-15 minutes.

Once it's done, test it:

```
npx serve .
```

Open localhost:3000 in Chrome. You should see a dark-themed habit tracker. Add a habit. Check it off. Refresh the page — it should persist.

Fixing What's Broken

First build is never perfect. Here are the three things you'll likely need to fix:

Problem 1: The toggle doesn't feel right.

Tell Claude:

The habit toggle needs haptic feedback. Add a subtle CSS animation on tap – scale to 0.95 then back to 1.0 over 150ms. Also add a checkmark SVG that animates in with a draw effect over 300ms when a habit is completed.

Problem 2: The streak calculation is wrong.

Tell Claude:

The streak counter is broken. A streak should count consecutive days where the habit was completed, ending yesterday or today. If today is incomplete, the streak should still show based on yesterday. If yesterday was missed, streak resets to 0 regardless of today's status. Fix the `calculateStreak` function in `storage.js`.

Problem 3: It doesn't look professional.

Tell Claude:

The UI needs polish:

1. Add a subtle gradient on the header (from #0A0A0A to #1A1A2E)
2. Habit cards should have a 1px border of `rgba(255,255,255,0.08)`
3. Add a floating action button (bottom-right, +) for adding new habits
4. The empty state should say "No habits yet. Tap + to start building streaks."
5. Add a stats section at the top: total habits, active streaks, completion rate today

Each fix takes 2-5 minutes. You're iterating on a working product, not building from scratch.

Hour 2: PWA Features + Polish (1:00 - 2:00)

Make It Installable (15 minutes)

Your `manifest.json` should already exist. Verify it has these fields:

```

{
  "name": "StreakPad",
  "short_name": "StreakPad",
  "description": "Track habits. Build streaks. No account needed.",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#0A0A0A",
  "theme_color": "#00FF88",
  "icons": [
    { "src": "/icons/icon-192.svg", "sizes": "192x192", "type":
      "image/svg+xml" },
    { "src": "/icons/icon-512.svg", "sizes": "512x512", "type":
      "image/svg+xml" }
  ]
}

```

Tell Claude to verify the service worker caches all assets:

Update sw.js to use a cache-first strategy. Cache these files on install:
 /, /index.html, /css/style.css, /js/app.js, /js/storage.js,
 /manifest.json
 Version the cache name as 'streakpad-v1' so we can bust it on updates.
 Add a fetch handler that serves from cache first, falls back to network.

Test installability: Open Chrome DevTools > Application > Manifest. You should see a green checkmark next to “Installable.” Click “Install” in the browser bar.

Add Notifications (15 minutes)

Add a daily reminder feature. When a user creates a habit, they can optionally set a reminder time. Use the Notification API (request permission on first use).
 Store reminder times in localStorage. Check every minute if any reminders should fire. Show a notification with the habit name and current streak count.
 No backend needed – this runs entirely in the service worker.

Add Data Export (10 minutes)

Add a settings page (gear icon in header) with:

1. Export data as JSON (downloads a file)
2. Import data from JSON (file picker)
3. Clear all data (with confirmation dialog)
4. App version number

This page should slide in from the right with a CSS transition.

Final Polish (20 minutes)

Final polish pass:

1. Add page transitions – when navigating to settings, slide the current view left
and slide settings in from right. 300ms ease-out.
2. Add a "weekly view" – a 7-column grid showing completion status for the past week.
Green squares for completed, dark gray for missed. Like GitHub's contribution graph.
3. Add an onboarding overlay for first-time users: "Welcome to StreakPad" with
3 quick tips (swipe through). Show once, set a flag in `localStorage`.
4. Make the FAB pulse gently on first visit until the user taps it.

Deploy It

```
npx surge ./ streakpad-app.surge.sh
```

Your app is now live at `streakpad-app.surge.sh`. Free. Instant. SSL included.

Open it on your phone. Tap “Add to Home Screen.” You now have a native-feeling app on your phone that you built in 2 hours.

What You Actually Built

Let's be specific about what you have:

- A PWA that installs on iOS and Android without an app store
- Offline functionality via service worker
- Local data persistence (no server costs)
- Push notification reminders
- Data import/export
- A polished UI with animations and transitions
- An onboarding flow

This is not a toy project. This is what \$5K freelance clients pay for. The difference is you built it in 2 hours and spent \$0.80 in API credits.

The Exercise

Before moving to Module 3, build a second app. Pick one:

1. **Pomodoro timer** with session logging and daily stats
2. **Expense splitter** for groups (like Splitwise but simpler)
3. **Daily journal** with mood tracking and streaks

Same process. Same 2-hour window. The goal is speed, not perfection. Ship ugly, fix later, validate first.

If your second app takes longer than 2.5 hours, re-read your CLAUDE.md. You're probably letting Claude over-engineer it.

Next module: Getting your app on a custom domain and accessible to real users.

Module 3: Deploy to Production — Surge, Netlify, Vercel, Custom Domains

What You'll Have After This Module

Your app live on the internet with a custom domain, SSL certificate, and automatic deployments. Three deployment options depending on what you need, each taking under 15 minutes.

Option 1: Surge.sh — Fastest Path to Live (5 minutes)

Surge is the fastest way to get a static site live. No account creation. No dashboard. Just a terminal command.

First Deploy

```
npm install -g surge  
cd your-project-directory  
surge ./ your-app-name.surge.sh
```

First time, it'll ask for an email and password. That creates your account. Every deploy after that is one command.

Your app is now live at `your-app-name.surge.sh`. Free. SSL included.

Custom Domain on Surge

Buy a domain. Namecheap is cheapest at \$8-12/year for a .com. Porkbun is \$9/year and has free WHOIS privacy.

Add a CNAME file to your project root:

```
echo "myapp.com" > CNAME
```

In your domain registrar's DNS settings, add:

Type: CNAME
Host: @

Value: na-east1.surge.sh

For the www subdomain:

Type: CNAME

Host: www

Value: na-east1.surge.sh

Deploy again:

```
surge ./ myapp.com
```

DNS propagation takes 5-30 minutes. After that, myapp.com serves your app with automatic SSL.

Surge Pricing

- Free tier: Unlimited projects, custom domains, SSL
- Surge Professional (\$30/month): Password protection, CORS headers, custom redirects

For MVPs, free tier is all you need. Don't pay until you're making money.

When to Use Surge

- Rapid prototyping (deploy in under 60 seconds)
- Static sites with no server-side logic
- When you want zero config and zero dashboard

Option 2: Netlify — Best for Growing Apps (10 minutes)

Netlify adds features Surge doesn't have: form handling, serverless functions, branch deploys, and a visual dashboard.

Setup

```
npm install -g netlify-cli  
netlify login
```

This opens a browser window to authenticate. One-time setup.

Deploy from Terminal

```
cd your-project-directory  
netlify deploy --prod --dir .
```

First time, it'll ask you to create a new site or link to an existing one. Choose "Create & configure a new site."

Your app is live at `random-name-12345.netlify.app`.

Custom Domain on Netlify

In the Netlify dashboard (`app.netlify.com`):

1. Go to your site > Domain settings > Add custom domain
2. Enter your domain (e.g., `myapp.com`)
3. Netlify gives you DNS records to add at your registrar

Option A — Use Netlify DNS (recommended): - Change your domain's nameservers to Netlify's (they provide 4 nameservers) - Everything's managed in one place - Automatic SSL provisioning

Option B — External DNS: - Add an A record pointing to `75.2.60.5` - Add a CNAME for `www` pointing to your Netlify subdomain - SSL still works but takes longer to provision

Auto-Deploy from Git

This is where Netlify shines. Connect your GitHub repo:

```
cd your-project-directory
git init
git add -A
git commit -m "initial commit"
gh repo create myapp --public --push
netlify link
```

Now every `git push` automatically deploys your site. Branch deploys give you preview URLs for testing before merging.

Netlify Functions (Serverless)

Need a backend endpoint? Create `netlify/functions/hello.js`:

```
exports.handler = async (event) => {
  return {
    statusCode: 200,
    body: JSON.stringify({ message: "Hello from the backend" }),
  };
};
```

This is available at `/.netlify/functions/hello`. No server to manage. 125K free invocations/month.

Use this for: Stripe webhooks, email sending, API proxying (hide your API keys from the client).

Netlify Pricing

- Free: 100GB bandwidth, 125K function invocations, 1 concurrent build
- Pro (\$19/month): 1TB bandwidth, password protection, analytics
- You won't need Pro until you're past \$500/month in revenue

When to Use Netlify

- Apps that need serverless functions
- Team projects with branch deploys
- When you want form handling without a backend
- When git-based auto-deploy matters

Option 3: Vercel — Best for Next.js and APIs (10 minutes)

Vercel is built by the Next.js team. If you're using Next.js, use Vercel. If you're using vanilla JS/HTML, Vercel still works but Surge or Netlify is simpler.

Setup

```
npm install -g vercel
vercel login
```

Deploy

```
cd your-project-directory
vercel --prod
```

First deploy asks a few questions (project name, framework, output directory). After that, `vercel --prod` is all you need.

Custom Domain on Vercel

```
vercel domains add myapp.com
```

Vercel gives you DNS records. Add them at your registrar. Same process as Netlify — either use Vercel's nameservers or add A/CNAME records.

Vercel Serverless Functions

Create `api/hello.js`:

```
export default function handler(req, res) {
  res.status(200).json({ message: "Hello from Vercel" });
}
```

Available at `/api/hello`. Vercel's function syntax is simpler than Netlify's, and cold starts are faster.

Edge Functions

Vercel's edge functions run at the CDN level — 50ms response times globally:

```
// api/fast.js
export const config = { runtime: 'edge' };

export default function handler(req) {
  return new Response(JSON.stringify({ fast: true }), {
    headers: { 'content-type': 'application/json' },
  });
}
```

Vercel Pricing

- Hobby (free): 100GB bandwidth, 100K function invocations
- Pro (\$20/month): 1TB bandwidth, 1M invocations, team features
- Free tier is generous enough for apps making up to \$1K/month

When to Use Vercel

- Next.js projects (zero config, it just works)
- When you need edge functions for global speed
- API-heavy apps where every endpoint is a serverless function

The Decision Matrix

Need	Use
Ship in 60 seconds, no account needed	Surge
Static site with custom domain	Surge
Need serverless functions	Netlify or Vercel
Git-based auto-deploy	Netlify or Vercel
Next.js app	Vercel
Form handling without backend	Netlify
Maximum simplicity	Surge

Domain Strategy for Multiple Apps

If you're shipping 10+ apps (Module 5), don't buy a domain for each one. That's \$100+/year in domains alone.

Strategy: Buy one domain, use subdomains.

myapps.com → portfolio/landing page
tracker.myapps.com → habit tracker
timer.myapps.com → pomodoro app
split.myapps.com → expense splitter

On Netlify/Vercel, each subdomain can point to a different project. One domain, unlimited apps.

Cost: \$10/year for the domain. \$0 for hosting. \$0 for SSL.

Post-Deploy Checklist

After every deploy, verify these five things:

1. **SSL works** — Visit <https://yourapp.com>. Green lock icon. No mixed content warnings.
2. **Mobile works** — Open on your phone. Test every interaction. Tap targets need to be 44x44px minimum.
3. **PWA installs** — On mobile Chrome, check if the “Add to Home Screen” prompt appears.
4. **Meta tags exist** — Open Graph tags for social sharing. Test with LinkedIn’s Post Inspector and Twitter’s Card Validator.
5. **Analytics tracking** — Add Plausible (\$9/month, privacy-friendly) or Umami (free, self-hosted). You need data before you can optimize.

Add these OG tags to your `index.html`:

```
<meta property="og:title" content="StreakPad – Build Better Habits">
<meta property="og:description" content="Track habits, build streaks,
    stay consistent. Free, offline, no account needed.">
<meta property="og:image" content="https://myapp.com/og-image.png">
<meta property="og:url" content="https://myapp.com">
<meta name="twitter:card" content="summary_large_image">
```

Create the OG image at 1200x630px. Use Figma (free) or tell Claude Code to generate an SVG and convert it.

Deployment Automation

Once you’ve deployed manually 3 times, automate it. Add a deploy script to your `package.json`:

```
{
  "scripts": {
    "deploy": "surge ./ myapp.com",
    "deploy:preview": "surge ./ preview-myapp.surge.sh"
  }
}
```

Now `npm run deploy` ships to production. `npm run deploy:preview` ships to a preview URL you can share for feedback.

Next module: Adding Stripe payments so your free app becomes a revenue-generating product.

Module 4: Monetization — Stripe, Paywalls, and Your First Dollar

What You'll Have After This Module

A working payment system in your app. Users can pay you money. That's the whole point of everything else you've built.

Why Stripe and Nothing Else

Stripe charges 2.9% + \$0.30 per transaction. On a \$5/month subscription, you keep \$4.56. On a \$50 one-time payment, you keep \$48.25.

Don't use Gumroad for in-app payments (10% fee). Don't use PayPal (dispute nightmares). Don't use Paddle unless you're selling to the EU and need them to handle VAT.

Stripe is the answer. Every time. Create your account at dashboard.stripe.com.

Architecture: Client-Only vs. Client + Serverless

Client-Only (Stripe Payment Links)

This is the fastest path. No backend code at all.

1. Go to Stripe Dashboard > Payment Links
2. Create a link for your product (\$5/month subscription or \$10 one-time)
3. Customize the checkout page (add your logo, description)
4. Copy the link
5. Put it behind your paywall button

```
// In your app
document.getElementById('upgrade-btn').addEventListener('click', () =>
{
  window.location.href = 'https://buy.stripe.com/your-payment-link';
});
```

After payment, Stripe redirects to your success URL. Add `?session_id={CHECKOUT_SESSION_ID}` to the redirect URL to verify payment.

Pros: 5 minutes to implement. Zero backend. **Cons:** Can't customize the checkout flow. Limited to Stripe's hosted page.

Client + Serverless (Full Control)

For more control, use Stripe Checkout with a serverless function. This is what you'll use for most real products.

Create a Netlify function at `netlify/functions/create-checkout.js`:

```
const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY);

exports.handler = async (event) => {
  if (event.httpMethod !== 'POST') {
    return { statusCode: 405, body: 'Method not allowed' };
  }

  const { priceId } = JSON.parse(event.body);

  const session = await stripe.checkout.sessions.create({
    payment_method_types: ['card'],
    line_items: [{ price: priceId, quantity: 1 }],
    mode: 'subscription', // or 'payment' for one-time
    success_url: `${process.env.URL}/success?session_id={CHECKOUT_SESSION_ID}`,
    cancel_url: `${process.env.URL}/`,
  });

  return {
    statusCode: 200,
    body: JSON.stringify({ url: session.url }),
  };
};
```

Set your environment variable in Netlify: Site Settings > Environment Variables > Add STRIPE_SECRET_KEY.

Frontend call:

```
async function startCheckout() {
  const response = await fetch('/.netlify/functions/create-checkout',
    {
      method: 'POST',
      body: JSON.stringify({ priceId: 'price_1234567890' }),
    });
  const { url } = await response.json();
  window.location.href = url;
}
```

The priceId comes from your Stripe Dashboard > Products > Create Product > Copy the price ID (starts with price_).

Paywall Strategies That Actually Convert

Strategy 1: Freemium with Feature Gating

Free users get 3 habits. Paid users get unlimited.

```
const MAX_FREE_HABITS = 3;
const isPaid = localStorage.getItem('paid_status') === 'true';

function canAddHabit() {
  const habits = getHabits();
  if (habits.length >= MAX_FREE_HABITS && !isPaid) {
    showUpgradeModal();
    return false;
  }
  return true;
}
```

The upgrade modal should show: - What they get (unlimited habits, data export, custom colors) - Social proof (“Join 200+ users building better habits”) - Price anchoring (“Less than a coffee per month”) - A clear CTA button that triggers startCheckout()

Strategy 2: Free Trial with Time Limit

Full access for 7 days. Then paywall.

```
function checkTrialStatus() {
  const installDate = localStorage.getItem('install_date');
  if (!installDate) {
    localStorage.setItem('install_date', Date.now().toString());
    return 'active';
  }

  const daysSinceInstall = (Date.now() - parseInt(installDate)) /
    (1000 * 60 * 60 * 24);
  const isPaid = localStorage.getItem('paid_status') === 'true';

  if (daysSinceInstall > 7 && !isPaid) {
    return 'expired';
  }
  return 'active';
}
```

When the trial expires, don't lock them out completely. Let them see their data but not add new entries. Loss aversion is your best conversion tool — they've already built streaks, they don't want to lose them.

Strategy 3: One-Time Purchase (Lifetime Access)

Best for tools that don't have recurring costs. Charge \$15-50 once.

This is the simplest model. No subscription management, no churn tracking, no dunning emails. User pays once, gets full access forever.

Store the purchase status after successful checkout:

```
// On your success page
const urlParams = new URLSearchParams(window.location.search);
const sessionId = urlParams.get('session_id');

if (sessionId) {
  // Verify with your serverless function
  const response = await fetch(`/.netlify/functions/verify-payment?
    session_id=${sessionId}`);
  const { paid } = await response.json();

  if (paid) {
    localStorage.setItem('paid_status', 'true');
    localStorage.setItem('purchase_date', Date.now().toString());
  }
}
```

The verification function:

```
const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY);

exports.handler = async (event) => {
  const { session_id } = event.queryStringParameters;

  const session = await stripe.checkout.sessions.retrieve(session_id);

  return {
    statusCode: 200,
    body: JSON.stringify({ paid: session.payment_status === 'paid' }),
  };
};
```

Pricing: What to Actually Charge

For PWAs and simple tools:

Model	Price Point	Works When
One-time	\$9 - \$29	Tool solves a one-time problem or has no server costs
Monthly	\$3 - \$9/month	Tool provides ongoing value (tracking, analytics, daily use)
Annual	\$29 - \$79/year	Offer 2 months free vs monthly to incentivize annual
Lifetime	\$29 - \$99	You want cash upfront and don't want to manage subscriptions

Rules: - Never price below \$5 for a one-time purchase. You'll attract complainers and refund requests. - Always offer annual pricing alongside monthly. 30-40% of users will choose annual. - Test pricing. Start at \$9/month. If conversion rate is above 5%, raise to \$12. Keep raising until conversion drops below 2%.

Handling Payments Without a Database

For simple apps, you don't need a database to track who's paid. Here's the pattern:

1. User pays via Stripe Checkout
2. Stripe redirects to your success page with a session ID
3. Your serverless function verifies the session with Stripe
4. You set `paid_status` in `localStorage`

The weakness: Users can clear `localStorage` and lose their purchase status. Two solutions:

Solution A: Stripe Customer Portal. Let users manage their own subscription and re-verify their status:

```
// Link to Stripe's hosted customer portal
const portalSession = await stripe.billingPortal.sessions.create({
  customer: customerId,
  return_url: 'https://myapp.com/settings',
});
```

Solution B: License keys. After purchase, generate a unique key, email it to the user, and store it in `localStorage`. If they clear storage, they re-enter the key. Use a serverless function to validate keys against a simple JSON file or KV store.

Solution C: Supabase (when you're ready for a database). Supabase gives you a Postgres database + auth for free (up to 50K monthly active users). When your app grows past `localStorage`, this is the next step.

Revenue Tracking

Don't rely on Stripe's dashboard alone. Create a simple revenue tracking system:

```
// In your admin/settings
async function getRevenue() {
  const response = await fetch('/.netlify/functions/revenue-stats');
  const { mrr, totalCustomers, churnRate } = await response.json();

  document.getElementById('mrr').textContent = `$$${mrr}`;
  document.getElementById('customers').textContent = totalCustomers;
  document.getElementById('churn').textContent = `${churnRate}%`;
}
```

The metrics that matter: - **MRR** (Monthly Recurring Revenue): How much you make per month from subscriptions - **Churn rate**: What percentage of subscribers cancel per month (below 5% is good, below 3% is great) - **LTV** (Lifetime Value): Average revenue

per customer over their lifetime. If $LTV > \$50$, you can afford to spend on acquisition. - **Conversion rate:** Free users who become paid users. 2-5% is normal. Above 5% means you should raise prices.

Stripe Test Mode

Before going live, test everything in Stripe's test mode. Use these test card numbers:

- 4242 4242 4242 4242 — successful payment
- 4000 0000 0000 0002 — declined card
- 4000 0000 0000 3220 — requires 3D Secure

Any future expiry date and any 3-digit CVC will work.

Toggle between test and live mode by switching your API keys. Test keys start with `sk_test_`, live keys start with `sk_live_`.

Go live only after you've tested: successful payment, failed payment, subscription cancellation, and the paywall correctly gating features.

Next module: Building a portfolio of 10+ apps and knowing which ones to kill, which ones to double down on.

Module 5: The Portfolio System — Ship 10 Apps, Kill 8, Scale 2

What You'll Have After This Module

A repeatable system for generating app ideas, building them in 48 hours, testing them with real users, and making data-driven decisions about which ones to kill and which ones to pour fuel on. This is how solo developers build \$10K/month income streams.

The Math That Makes This Work

Most apps fail. This isn't pessimism — it's the base rate. Out of every 10 apps you launch:

- 5-6 will get zero traction (nobody cares)
- 2-3 will get some users but won't monetize
- 1-2 will make money

If each app takes you 6 months and \$5K, you can't afford to fail. But if each app takes 48 hours and \$2 in API credits, you can fail 8 times and still win.

Your job is not to predict which app will work. Your job is to ship fast enough that the numbers play out in your favor.

Phase 1: Idea Generation (The Hit List)

You need 10 app ideas. Not 10 good ideas — 10 ideas that meet three criteria:

1. **Someone is already paying for a solution.** Search the App Store, Product Hunt, or Gumroad. If nobody's paying for it, the demand doesn't exist.
2. **You can build an MVP in under 48 hours.** If it needs user accounts, real-time data, or complex algorithms, it's too complex for a first pass.
3. **The value is obvious in 10 seconds.** If you need a paragraph to explain what it does, nobody will try it.

Here are 10 proven categories with specific examples:

#	Category	Example App	Who Pays For This
1	Habit tracking	StreakPad (you built this)	47 paid apps on App Store, \$3-\$10/mo
2	Pomodoro/Focus	FocusBlock — timer + website blocker	Forest app: \$4, 10M+ downloads
3	Expense splitting	SplitEasy — simpler Splitwise	Splitwise: freemium, millions of users
4	Meal planning	MealMap — weekly meal planner + grocery list	Mealime: \$6/mo, 1M+ downloads
5	Invoice generator	QuickBill — freelancer invoices in 30 seconds	Invoice Ninja, Wave (millions in revenue)
6	Bookmark manager	StackMark — save + tag + search bookmarks	Raindrop.io: \$3/mo, 500K users
7	Daily journaling	OneLine — one sentence per day journal	Day One: \$5/mo, Apple Design Award
8	Countdown timer	EventPulse — countdowns to things you care about	Multiple apps charging \$2-\$5
9	Color palette	PaletteForge — generate + save color palettes	Coolors Pro: \$3/mo
10	QR code generator	QRForge — styled QR codes with logos	QR Tiger: \$7/mo

Write your hit list. 10 apps. For each one, note: - The existing paid competitor - What they charge - What you'd build differently (simpler, faster, offline-first, better design)

Phase 2: The 48-Hour Sprint

You already know how to do this from Module 2. Here's the optimized workflow:

Hours 0-4: Build the core loop.

The core loop is the one thing your app does. For a habit tracker, it's: add habit, check it off, see streak. Everything else is decoration.

Tell Claude Code exactly what the core loop is. Don't describe features — describe the sequence of user actions.

Hours 4-8: Add the paywall and polish.

Apply one of the paywall strategies from Module 4. Add OG tags for sharing. Polish the UI with the prompts from Module 2.

Hours 8-10: Deploy and create landing context.

Deploy to Surge/Netlify. Write a one-paragraph description. Take 3 screenshots (mobile). You're ready to show it to people.

Hours 10-48: Get 20 people to try it.

This is the hard part. Not because it's technically difficult, but because most builders hide behind "it's not ready yet." It's ready. Ship it.

Where to find 20 testers: - Reddit: Find the subreddit for your app's category. r/productivity, r/freelance, r/cooking, r/webdev. Post a Show HN-style post: "I built X in a weekend, looking for feedback." - Twitter: Post a 30-second screen recording. Tag relevant accounts. - Discord: Most niches have Discord servers. Search Disboard.org. Join and share in the appropriate channel. - Indie Hackers: Post in the "Show IH" section. This audience specifically wants to see what you built.

Phase 3: The Kill/Scale Decision (Day 7)

After 7 days, you have data. Here's how to read it:

Kill Triggers (stop working on this app immediately)

- **Zero organic traffic after sharing in 5+ places.** If people saw it and didn't click, the idea doesn't resonate.
- **High bounce rate (>80%) with >50 visitors.** People clicked but left immediately. The value proposition failed.
- **Zero repeat usage.** People tried it once and never came back. The core loop isn't sticky.
- **Negative feedback about the concept, not the execution.** "I wouldn't use this" is different from "this is buggy." The first is a kill signal. The second is a fix signal.

When you kill an app, don't delete it. Archive it. You might cannibalize the code later, or the market might shift.

Double-Down Triggers (pour fuel on this app)

- **Repeat usage without prompting.** If people come back without you reminding them, you've hit something.
- **Unprompted sharing.** Someone shared your app with someone else. This is the strongest signal.
- **Feature requests.** People wanting MORE from your app means the core value is right and they want to go deeper.
- **Any payment.** Someone gave you money. Even \$3. That's validation that can't be faked.
- **"How do I get my data back?"** Someone asking about data portability means they've put real data into your app. They're invested.

The Conversation You Have With Yourself

App: StreakPad

Users (7 days): 43

Return users: 12 (28%)

Feature requests: 4

Revenue: \$0

Shares: 2

Verdict: DOUBLE DOWN.

Why: 28% return rate is exceptional for a free tool.

Feature requests signal engagement.

Next step: Add the paywall (Strategy 1, freemium).

Timeline: This weekend.

App: QRForge

Users (7 days): 67

Return users: 3 (4%)

Feature requests: 0

Revenue: \$0

Shares: 0

Verdict: KILL.

Why: High traffic, near-zero retention.

QR codes are a one-time use tool – wrong model for recurring.

Not worth competing with free QR generators.

Archive and move on.

Phase 4: Scaling What Works

You've killed 8 apps and found 2 with traction. Now what?

Week 2-4: Feature Depth

Add the features people asked for. Every feature should take less than 4 hours to build. If it takes longer, break it into smaller pieces.

Priority order: 1. Features that increase retention (keep people coming back) 2. Features that justify the price (make the paywall feel fair) 3. Features that reduce churn (keep paying users from leaving)

Features that DON'T matter at this stage: - Social features (sharing, profiles, friends) - Design system overhauls - Performance optimization (unless the app is literally slow) - Multi-language support

Month 2: Distribution

Your app works and people pay for it. Now get more people.

SEO (free, slow): Create a landing page with proper H1, H2, meta description. Target long-tail keywords like “free habit tracker app no account” or “simple invoice generator freelancers.” Use Ubersuggest (free plan) to find keywords with >100 monthly searches and <30 difficulty.

Content (free, medium speed): Write 3-5 blog posts answering questions your target users search for. “How to build a daily habit routine” → links to your habit tracker. “How to invoice clients as a freelancer” → links to your invoice tool. Host the blog on the same domain.

Product Hunt (free, fast spike): Launch on Product Hunt on a Tuesday or Wednesday. Prepare screenshots, a 1-minute demo video, and a compelling tagline. Aim for top 5 of the day. One good PH launch can drive 1,000-5,000 users in a week.

Paid ads (not yet): Don't run ads until your conversion rate is stable and your LTV is clear. If you're converting 3% of visitors at \$5/month with 4-month average retention, your LTV is \$20. You can afford to spend \$5-\$8 per user acquisition. Not before.

Month 3+: The Compound Effect

If you ship 10 apps every quarter: - Quarter 1: 10 shipped, 2 survive = 2 revenue streams - Quarter 2: 10 more shipped, 2 survive = 4 revenue streams - Quarter 3: 10 more, 2 survive = 6 revenue streams

Each surviving app might make \$200-\$2,000/month. With 6 apps at an average of \$500/month, you're at \$3K/month from a portfolio of simple tools that each took 48 hours to build.

The apps that hit \$1K/month get a second developer (you hire a freelancer on Upwork for \$15/hr to add features while you keep shipping new ones).

The Portfolio Tracker

Track your apps in a simple spreadsheet:

App	Launch Date	Status	Users (7d)	Return Rate	MRR	Decision
StreakPad	Mar 1	Active	43	28%	\$47	SCALE

App	Launch Date	Status	Users (7d)	Return Rate	MRR	Decision
QRForge	Mar 8	Killed	67	4%	\$0	ARCHIVED
FocusBlock	Mar 15	Testing	-	-	-	PENDING

Update weekly. The data makes the decisions, not your emotions. If an app isn't performing after 14 days, kill it no matter how much you like the idea.

The One Rule

Ship speed is your only competitive advantage. Funded teams have more resources, better designers, bigger networks. But they can't ship 10 apps in a quarter. They'll spend that quarter in planning meetings debating the color of the signup button.

You ship today. You learn tomorrow. You scale next week. That's the whole system.